Laboratory 2 - Cluster Analysis

Clustering is a set of techniques used to partition data into groups (called clusters). Clusters are loosely defined as groups of data objects that are more similar to other objects in their cluster than they are to data objects in other clusters. Clustering helps identify two qualities of data: meaningfulness and usefulness.

TASKS TO BE CARRIED OUT

1. Cluster using **K-Means** algorithm following datasets: 1_S-sets, 4_Nested-datasets and 2_Birch-sets; select one file from each directory. Find the correct number of clusters - run several K-Means, increment k with each iteration, and record the silhouette coefficient. Plot the final clusters with different colors. Discuss the results. - 2 pkt

3. Cluster using **DBSCAN** method following datasets: 1_S-sets, 4_Nested-datasets and 2_Birch-sets; select one file from each directory. Plot the final clusters with different colors. Discuss the results. - 2 pkt

4. Compare used clustering methods (also **Agglomerative Clustering** from Laboratory 3) regard to the dataset. - 1 pkt

USEFUL INFORMATION

For **K-Means Clustering** use sklearn.cluster package.
First standardize features using StandardScaler() method from sklearn.preprocessing:

```
    myscaler = StandardScaler()
    scaled_features = myscaler.fit_transform(dataset)
```

Then run K-Means clustering in loop for a range of cluster numbers

```
    k_means = KMeans(n_clusters=k, **kmeans_args)
        k_means.fit(scaled_features)
```

where:

```
    kmeans_args = {
        "init": "random",
        "n_init": ??,
        "max_iter": ??,
        "random_state": ??,
    }
# where KMeans(n_clusters=number of clusters to form,
#     init=method for initialization,
#     n_init=number of time the k-means algorithm will be run with different centroid seeds,
#     max_iter=maximum number of iterations of the k-means algorithm for a single run,
#     random_state=determines random number generation for centroid initialization)
```

In the loop calculate also silhouette coefficient using silhouette_score method from sklearn.metrics.

```
    silhouette_score(scaled_features, k_means.labels_)
```

You can check the final locations of the centroid

k_means.cluster_centers_

You can check the number of iterations required to converge

k_means.n_iter_

For **DBSCAN** clustering use sklearn.cluster package.
First standardize features using StandardScaler() method from sklearn.preprocessing.

Then use db=DBSCAN(eps=??, min_samples=??).fit(scaledData)

Estimate number of clusters

n_clusters_ = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)

Estimate number of noise points

n_noise_ = list(db.labels_).count(-1)

Calculate silhouette coefficient